kylinpkg

在银河麒麟桌面操作系统 V10 SP1 上，按需从**任意 apt 源配置**、**任意架构**下载 deb 包及其依赖，并生成可直接给目标机使用的离线 apt 源。

核心特点：所有 apt 操作都在隔离的 `Dir` 里进行，**不会修改本机的** `/etc/apt`、`/var/lib/dpkg`、`/var/lib/apt`。

当前版本：**0.3.4**。版本记录见 CHANGELOG.md，
开发/测试/发布流程见 CONTRIBUTING.md；
PDF 版使用手册见 docs/kylinpkg-manual.pdf
（由 `packaging/build-doc-pdf.py` 从本文件生成，GUI“帮助”菜单可直接打开）。

apt 到底依赖哪些信息

apt 求解"要装哪些包、下哪些包"时，输入是四类信息：

输入	位置	作用
源索引	`/var/lib/apt/lists/*_Packages`	源里有哪些包、版本、依赖、文件名、哈希
已安装状态	`/var/lib/dpkg/status`	哪些包已存在、版本、架构
源与版本策略	`sources.list` / `sources.list.d` + `preferences`	从哪取、选哪个版本、pin 优先级
架构集合	`APT::Architecture(s)`	为哪个架构求解

`/var/cache/apt/archives` 只是已下载 `.deb` 的缓存，不参与依赖求解。

因此本工具为每个 `profile × 架构` 建一个隔离 root，把上述四类输入全部指到自己的目录，再调用本机 apt-get 求解、下载、校验。

需要提前收集什么

- **源配置**：需要。按目标小版本保存为 profile（含 suite、组件、pin、keyring）。
- **dpkg status**：
- 做**全量闭包**（目标机可能是裸系统、或要建离线仓库）：用**空基线**，apt 会把 libc6 等基础依赖全部算进来，**不需要**每个版本的 status。
- 做**增量补包**（目标机已经是某小版本默认安装）：需要该版本的基线 status，且必须来自**对应版本的干净 VM/镜像**，不是本机当前状态（本机可能已经装了额外软件、或版本已被更新）。
- 麒麟的"小版本"通常不是单独一套源，而是**基础 suite + 叠加 suite + 修复合集**，例如：
- `10.1-kylin`（基础）
- `10.1-kylin-2503`（叠加更新）
- `10.1-2503-bugfix-limit`（限定修复）

- partner：`archive2.kylinos.cn/.../partner/V10-SP1`
  只挂叠加 suite 会缺基础包（实测 `libgcc-s1` 就会解析失败）。

环境要求

- 银河麒麟 V10 SP1 / 其他 Debian、Ubuntu 系发行版
- Python 3.8+
- `apt-get`、`dpkg`、`dpkg-deb`、`apt-ftparchive`（或 `dpkg-scanpackages`）
- 可选：`ssh`（从目标机抓取配置）、`tar`

不需要 root；以普通用户运行即可。状态目录默认 `~/.local/share/kylinpkg`。

```
git clone https://github.com/MoHaO-bot/kydesktop-tools.git
cd kydesktop-tools
./bin/kylinpkg --version
./bin/kylinpkg doctor
```

也可以把 `bin/kylinpkg` 软链到 `~/bin` 或 `/usr/local/bin`。

安装 deb（推荐）

从 Releases 下载最新 deb 直接安装：

```
sudo dpkg -i ./kylinpkg_0.3.4_all.deb
```

也可以在源码目录自行构建（产物在 `dist/`，不进版本库）：

```
./packaging/build-deb.sh
sudo dpkg -i ./kylinpkg_0.3.4_all.deb
```

安装后的路径：

内容	路径
主程序	`/opt/kylinpkg/`
命令行	`/usr/bin/kylinpkg` → `/opt/kylinpkg/bin/kylinpkg`
图形界面	`/usr/bin/kylinpkg-gui` → `/opt/kylinpkg/bin/kylinpkg-gui`
桌面文件	`/usr/share/applications/kylinpkg.desktop`
图标	`/usr/share/icons/hicolor/scalable/apps/kylinpkg.svg`
收集脚本	`/opt/kylinpkg/collector/kylinpkg-collect.sh`

默认下载输出目录是 `/tmp/kylinpkg-out/<profile>-<架构>`，可用 `-o` 或环境变量 `KYLINPKG_OUTPUT_DIR` 覆盖。运行状态仍保存在 `~/.local/share/kylinpkg`。

卸载：`sudo apt remove kylinpkg`。

图形界面

```
./bin/kylinpkg-gui
# 或
./bin/kylinpkg gui
```

GUI 与 CLI 共用同一个状态目录（默认 `~/.local/share/kylinpkg`），
在 GUI 里导入的 profile/baseline 可以直接被命令行使用，反之亦然。

界面分三块（顶部另有 **“导入其他主机配置”**、**“任务记录”** 和 **“清理缓存”** 按钮）：

菜单栏的 **帮助 → 使用手册** 会用系统默认程序打开 PDF 版使用手册
（`docs/kylinpkg-manual.pdf`，随 deb 一起安装到 `/opt/kylinpkg/docs/`）。

顶部的 **“任务记录”** 打开下载历史对话框：每次下载（含失败/中断）都会留痕，
可查看任务的时间、状态、profile、架构、软件包与输出目录，支持
重新下载（按原配置重跑）、**应用到当前参数**（自动填入下载表单，
双击记录同效）和删除单条记录。

1. **apt 源配置（profile）**：按钮行保持 `从预设创建`、`从本机导入`、`编辑源配置`、`更多`、`删除` 一行显示；`导入源配置`（自定义 `sources.list` / `sources.list.d` /preferences/keyring/auth.conf）和 `从 rootfs 导入` 收在 **“更多”** 菜单里。支持同名资源的管理和删除。
2. **dpkg 初始状态（baseline）**：`导入 status 文件` 支持选择目标机的 `/var/lib/dpkg/status`，或从本机/rootfs/空基线导入，并命名保存。
3. **下载参数**：选择 profile + baseline（或“全量闭包”），填写软件包名（空格或逗号分隔），可用 **“选择版本”** 查看 `apt policy` 里的所有版本；
   默认勾选 **“下载依赖”**，取消后只下载当前包本身；选择输出目录，然后：
   - `解析计划`：只做依赖求解，在下方日志里显示包清单、版本、大小；
   - `下载`：复用同一套隔离 apt root，执行 `apt-get install --download-only`，
   校验哈希、生成 `manifest.json`，可选择同时生成离线 apt 源。

架构是下拉选择：`amd64`、`arm64`、`mips64el`、`loongarch64`、`i386`，
以及 `amd64,i386` 组合。已有的 profile 若带其他架构，
选中时会动态显示，但默认选项只保留上述几个。

解析/下载在后台线程执行，界面不会卡住；日志区会显示 apt 的解析结果、下载结果和警告。
任务运行时旁边会出现 **“中断”** 按钮：点击会终止正在运行的 apt 及其子进程，
不会修改本机；已下载的部分文件会保留，下次可继续/重试。

命令行下按 `Ctrl+C` 也会中断当前 apt 子进程。
图形界面需要 `python3-tk`（麒麟一般已自带）；没有图形环境时请使用 CLI。

安装 deb 后会自动出现在开始菜单；源码运行时直接用 `./bin/kylinpkg-gui`。
桌面文件源码在 `packaging/kylinpkg.desktop`，安装后位于
`/usr/share/applications/kylinpkg.desktop`。

从原始主机收集（bundle）

如果无法直接访问目标版本的原始主机，可以先在原始主机上收集一个包，
再拿回下载器一键导入。收集脚本在 `collector/kylinpkg-collect.sh`。

```
# 1) 拷到原始主机并运行（建议 sudo，能读到 auth.conf 等 600 文件）
scp collector/kylinpkg-collect.sh user@target:/tmp/
ssh user@target 'sudo bash /tmp/kylinpkg-collect.sh -o /tmp'
# 输出：/tmp/kylinpkg-bundle-<hostname>-<时间>.tar.gz

# 2) 拿回下载器
scp user@target:/tmp/kylinpkg-bundle-*.tar.gz ./

# 3) 先看内容
./bin/kylinpkg bundle inspect ./kylinpkg-bundle-xxx.tar.gz

# 4) 一键导入 profile + baseline（默认按主机名命名并绑定同架构基线）
./bin/kylinpkg bundle import ./kylinpkg-bundle-xxx.tar.gz \
  --profile sp1-2303 --baseline sp1-2303-default

# 5) 之后就可以正常解析/下载
./bin/kylinpkg resolve -p sp1-2303 zlib1g
```

也可以直接在 GUI 顶部点 **“导入其他主机配置”**，选择 tar.gz 并填写名称；
对话框分「选择收集包」「导入设置」两步，文件选择窗口默认打开当前用户的
桌面目录；对话框里还提供 **“下载收集脚本”** 和 **“使用手册”** 按钮，可把
`collector/kylinpkg-collect.sh` 和说明文档另存到任意位置。

收集包包含：

- 完整 `rootfs/var/lib/dpkg/status`
- `rootfs/etc/apt` 下的 sources、sources.list.d、preferences、trusted keyrings
- `Signed-By` 引用的 keyring（复制到包内，导入时自动改写为 profile 内路径）
- `meta/` 中的 os-release、架构、apt/dpkg 版本、uname、包列表
- `SHA256SUMS` 与 `bundle.json`

注意：

- 默认**不收集** `/etc/apt/auth.conf`（私有源凭据）。需要时收集端加 `--with-auth`，导入时 `bundle import` 会自动带上，`profile import --with-auth` 也支持。
- 收集包可能包含私有源地址/凭据，脚本会把文件权限设为 600，请妥善保管；用 `sudo` 运行时脚本会把 tar.gz 的属主改回 `SUDO_USER`，方便直接 scp。
- 跨架构使用：`bundle import ... --profile xxx-arm64 --arch arm64 --no-bind`（原基线是 amd64 时不能绑定到 arm64 profile）。
- 更多细节见 `collector/README.md`。

内置源预设（45 个）

数据解析自《银河麒麟桌面外网源.xls》——**在线推送使用的正式源（外网）**，
纯文本副本保存在 `docs/kylin-desktop-sources.md`，已内置到 `kylinpkg/presets.py`。
共 45 个预设 / 218 条 `deb` 源行，其中 12 条附带 `kylin.pref` 优先级配置；
另有 3 条为 2026-10 现场实测预设（见下文「现场实测预设」）。

```
# 查看全部预设
./bin/kylinpkg preset list

# 查看某个预设的源地址和注意事项
./bin/kylinpkg preset show xc-2503

# 从预设创建一个可编辑的 profile
./bin/kylinpkg preset create xc-2503 --profile xc-2503 --arch amd64
# 也可以直接绑定已有 baseline
./bin/kylinpkg preset create xc-2503 --profile xc-2503 \
  --arch amd64 --baseline xc-2503-default
```

图形界面：源配置面板点 **“从预设创建”**，左侧选预设、右侧看源预览，
填写名称/架构/基线后创建；列表里“优先级”一列标出哪些预设带 `kylin.pref`。

预设分组（按 xls 的产线版本划分）：

分组	数量	ID
XC 产线（通用版）	9	`xc-2107`、`xc-2203`、`xc-2303`、`xc-2303u2`、`xc-2403u2`、`xc-2503`、`xc-2203-实机`、`xc-2303-实机`、`xc-2503-实机`（后 3 条为现场实测）
HWE 硬件启用栈	10	`hwe-2107`、`hwe-2203`、`hwe-2303`、`hwe-2303u2`、`hwe-2403u2`、`hwe-2503`、`hwe-pp-2303`、`hwe-pp-2303u2`、`hwe-pp-2403u2`、`hwe-pp-2503`
Wayland · 麒麟 990	6	`wl-2107-990`、`wl-2203-990`、`wl-2303-990`、`wl-2303u2-990`、`wl-2403u2-990`、`wl-2503-990`
Wayland · 麒麟 9006C	6	`wl-2107-9006c`、`wl-2203-9006c`、`wl-2303-9006c`、`wl-2303u2-9006c`、`wl-2403u2-9006c`、`wl-2503-9006c`
Wayland · 盘古 M900（Pangwu）	3	`wl-2303u2-m900`、`wl-2403u2-m900`、`wl-2503-m900`
Wayland · 麒麟 9000C	6	`wl-2403-9000c`、`wl-2403u1-9000c`、`wl-2403u2-9000c`、`wl-2503-9000c`、`wl-pangux-a-2403`、`wl-pangux-b`
Wayland · 华为其他平台	3	`wl-flemingx`、`wl-panguy`、`wl-2107-panguw`
V10 基础版（非 SP1，历史）	2	`v10-2107`、`v10-2203`

架构缺省规则：

- 预设只对应单一平台（Wayland/Kirin/盘古）时用该架构（`arm64`）；
- 预设列出多个架构时，优先选本机架构（例如 amd64 机器选 `amd64`）；
- 预设没有架构标注时，用本机 dpkg 架构；
- 始终可以用 `--arch` 或 GUI 下拉框覆盖。

2503 系列的优先级配置

2503 各产线（`xc-2503`、`hwe-2503`、`hwe-pp-2503`、以及 Wayland 2503 各芯片分支）
和华为 Pangux/Flemingx/Panguy 平台在 xls 里附带了
`/etc/apt/preferences.d/kylin.pref` 的 `Pin-Priority` 设置。
这些预设带有 `preferences` 字段，创建 profile 时会自动写进
`preferences.d/kylin.pref`，并给出一条警告提示确认 Pin-Priority 与本机系统版本匹配。
创建后可在“查看/编辑源配置”里直接修改。

现场实测预设（`*-实机`，2026-10 新增）

来自用 `collector/kylinpkg-collect.sh` 在 7 台现场主机（均为 XC 通用版）收集的
配置包：sp1-2203 / 2303 / 2403 / 2503 各 x86、arm64 一台。
已按重复配置去重：

- **`xc-2503-实机`**：sp1-2503 的 x86 / arm64 两台配置一致，合并为一条
  （amd64 + arm64，含实测 `kylin.pref`：550 一档）；
- **`xc-2303-实机`**：sp1-2303 的 x86 / arm64 两台配置一致，合并为一条
  （amd64 + arm64）；
- **sp1-2403**（x86 / arm64）与官方清单 `xc-2403u2` 逐行一致，不再单列；
- **`xc-2203-实机`** / **`xc-2303-实机`**：现场"源管理器"标准配置，
  archive2 走 `DEB/KYLIN_DEB` 入口、无 bugfix-limit suite，与官方清单
  `xc-2203` / `xc-2303` 并存，按现场机器实际状态选用；
- sp1-2203 现场重复的 pref、sp1-2303 的单机本地源（`/opt/apps-third/`）
  未收录，差异细节见 `docs/kylin-desktop-sources.md` 第九节。

与旧版源汇总的差异

旧的《银河麒麟桌面版软件源地址汇总.md》已过时，本次按 xls 整体重建：

- 预设 ID 全部改为 xls 的产线命名（`xc-*` / `hwe-*` / `wl-*`），
  旧的 `sp1-*` ID 不再存在；
- 2503 的 bugfix suite 以 xls 为准统一为 `10.1-2503-bugfix-limit`
  （旧文档写的 `10.1-2503-bugfix` 已被取代）；现场实测机器实际在用
  非 `-limit` 版本，故 `xc-2503-实机` 如实保留 `10.1-2503-bugfix`；
- 新增 `*-bugfix-limit`、`*-update3`（2303 update2 的更新层）、
  `10.1-2403-wayland-*` 系列 suite；
- 新增 HWE-PP、盘古 M900、麒麟 9000C、华为 Pangux/Flemingx/Panguy/Panguw 等产线；
- HWE 源不再使用 `archive2` 的 `KY-V10-SP1-$(ARCH)` 桌面源，改用 `KYLIN-ALL` 的
  `*-hwe-updates` / `*-hwe-bugfix-limit`；
- 只保留 2 条 V10 非 SP1（10.0 系）历史预设，新清单不再收录它们。

每次修改 `docs/kylin-desktop-sources.md` 后运行测试，
`tests/test_presets.py` 会逐行校验预设与文档一致。

查看和编辑源配置

图形界面：选中一个 profile，点 **“编辑源配置”**。左侧是文件列表，
右侧是文本编辑器，可 **保存 / 还原 / 新建源文件 / 删除 / 验证**。
保存时会：

- 校验 `.list` / `.sources` 语法；
- 拒绝删除最后一个源（避免 profile 变成空壳）；

- 重新计算文件清单、源摘要、警告和索引指纹，下次解析会自动刷新 apt 索引；
- keyring 文件只读（由导入/收集包管理）。

命令行：

```
# 列出文件（可编辑=是/否）
./bin/kylinpkg profile files sp1-2503

# 查看
./bin/kylinpkg profile cat sp1-2503 sources.list
./bin/kylinpkg profile cat sp1-2503 --all

# 用 $EDITOR 交互编辑
./bin/kylinpkg profile edit sp1-2503 sources.list

# 用文件/管道覆盖
./bin/kylinpkg profile set sp1-2503 sources.list --from ./new-sources.list
cat ./preferences | ./bin/kylinpkg profile set sp1-2503 preferences

# 删除或清空（删除最后一个源会被拒绝）
./bin/kylinpkg profile rm-file sp1-2503 sources.list.d/extra.list
```

可编辑文件：`sources.list`、`sources.list.d/*.list|*.sources`、`preferences`、`preferences.d/*`、`auth.conf`、`auth.conf.d/*`。`keyrings/*` 和 `profile.json` 不可编辑。`profile show --raw` 仍可用于一次性打印全部文本文件。

两个一等公民

profile：一套 apt 源配置

支持四种导入方式，源文件原样保存：

```
# 1) 从本机 /etc/apt 抓取
./bin/kylinpkg profile import sp1-2503-x86 --from-system --arch amd64,i386

# 2) 从干净目标机抓取（需要 SSH 免密）
./bin/kylinpkg profile import sp1-2503-arm64 --from-ssh mohao@172.16.101.137 --arch arm64

# 3) 从解开的 rootfs 抓取
./bin/kylinpkg profile import sp1-2503-img --from-rootfs /mnt/kylin-rootfs --arch amd64

# 4) 从导出的 sources 文件/目录导入
./bin/kylinpkg profile import sp1-2503-custom \
  --sources-file ./sources.list \
  --sources-dir ./sources.list.d \
  --preferences-dir ./preferences.d \
  --keyring ./kylin-archive-keyring.gpg \
  --arch amd64,i386
```

- `.list` 与 deb822 `.sources` 都支持。
- `--from-system/ssh/rootfs` 会同时抓取 `preferences`、`preferences.d` 和 `trusted.gpg*`。
- `--capture-baseline NAME` 可以在抓源的同时抓 `dpkg status`，一次完成 profile + baseline。
- `Signed-By` 引用的 keyring 会被复制进 profile 并改写路径（`keyrings/signed-by/`），便于跨机器复用；源里内嵌的凭据请用 `--with-auth` / `--auth-file` 显式导入。
- 架构可以写 `x86_64` / `aarch64` 别名，工具会归一成 `amd64` / `arm64`。
- 还提供按 Kylin suite 快捷生成的 `profile add`（适合源结构已知时）：

```
./bin/kylinpkg profile add sp1-2503 --kylin-suite 10.1-kylin-2503 --arch arm64
```

baseline：一份目标机 dpkg status

```
# 从目标机导出的 status 文件
./bin/kylinpkg baseline import sp1-2503-default --status-file ./status.sp1-2503

# 从目标机/ rootfs / 本机直接抓
./bin/kylinpkg baseline import sp1-2503-default --from-ssh mohao@172.16.101.137
./bin/kylinpkg baseline import sp1-2503-default --from-rootfs /mnt/kylin-rootfs
./bin/kylinpkg baseline import host-current --from-system
```

导入时会校验 Deb822 结构，统计已安装包数和架构；如果 baseline 中有 profile 未启用的架构，解析时会直接报错，避免"看起来成功、实际结果不对"。

> baseline 必须是完整的 `/var/lib/dpkg/status`（含 Version/Architecture/Status/依赖字段）。
> `dpkg --get-selections` 只有包名和状态，不能用于 apt 依赖求解。
> 跨架构下载时 baseline 也必须同架构：amd64 的基线不能用于 arm64；此时用 `--full`，或导入 arm64 目标机的 status。

内置现场基线（`sp1-*`，2026-10 新增）

随包分发 7 份 dpkg 初始状态快照（gzip 存储于 `kylinpkg/resources/baselines/`），
来自上面「现场实测预设」同一批 7 台 XC 通用版现场主机的收集包：

内置基线	已安装包	架构	内核（linux-generic）	对应源配置预设
`sp1-2203-x86`	1849	amd64	5.4.18.53.42	`xc-2203-实机`
`sp1-2303-x86`	1951	amd64	5.4.18.85.74	`xc-2303-实机`
`sp1-2303-arm64`	1943	arm64	5.4.18.85.74	`xc-2303-实机`
`sp1-2403-x86`	2059	amd64	5.4.18.110.99	`xc-2403u2`
`sp1-2403-arm64`	2052	arm64	5.4.18.110.99	`xc-2403u2`
`sp1-2503-x86`	2093	amd64	5.4.18.142.131	`xc-2503-实机`
`sp1-2503-arm64`	2084	arm64	5.4.18.142.131	`xc-2503-实机`

注意 2503 的 x86 / arm64 两份**包列表不同**，不能互相替代（这一点与源配置
不同——源配置两台一致合并为一条预设）；2303 / 2403 的 arm64 快照同样与
对应 x86 快照包列表不同。

```
# 启动时已自动导入，直接可见可用（来源标记为“内置”）
./bin/kylinpkg baseline list

# 如需换个名字再导一份，或删除后手工恢复
./bin/kylinpkg baseline import my-2503-arm64 --builtin sp1-2503-arm64
```

内置基线在 CLI / GUI **启动时自动导入**（幂等）：state 里缺哪条补哪条，
列表中与手工导入的 baseline 无任何区别，可直接绑定 profile、参与解析。
删除规则：你手动删除的内置基线不会被再次导入（同步标记记录在 state 的
`baselines/.builtin-sync.json`）；内置数据随包更新后会自动覆盖刷新；
启动前已手工导入的同名基线不会被覆盖。

GUI：无需额外操作，baseline 列表里直接就有这 7 条。
配合同名实机预设（如 `xc-2503-实机` + `sp1-2503-arm64`）即可在离线环境
还原一台现场主机的完整 apt 源 + 已装包状态。

新增内置基线的约定：把收集包里 `rootfs/var/lib/dpkg/status` 以 gzip 存为 `kylinpkg/resources/baselines/<名称>/status.gz`，并写一份同格式 `meta.json`（含 `name`、`status_sha256`、`installed_count`、`architectures`、`source_bundle` 等字段，可参考现有条目），`tests/test_builtin_baselines.py` 会校验清单与校验和。后续新增预设按同样方式扩充目录即可，无需改代码。

绑定与覆盖：

```
./bin/kylinpkg profile bind sp1-2503-x86 --baseline sp1-2503-default
./bin/kylinpkg profile bind sp1-2503-x86 --baseline empty   # 解除绑定，回到全量闭包
```

选择版本与多版本下载

apt 的 `policy` 里常能看到一个包有多个版本（基础源 / update / bugfix / 不同 suite）。
现在可以：

```
# 查看某个 profile/架构下该包的所有版本、优先级、是否可下载、来源
./bin/kylinpkg versions -p sp1-2503 zlib1g

# 只要某个版本
./bin/kylinpkg download -p sp1-2503 -o ./out zlib1g=1:1.2.11.dfsg-2kylin1.5k0.2

# 同时下载同一软件包的多个版本（会分别解析依赖，最后合并）
./bin/kylinpkg download -p sp1-2503 -o ./out --repo \
  zlib1g=1:1.2.11.dfsg-2kylin1.5k0.2 \
  zlib1g=1:1.2.11.dfsg-2kylin1.5k0.3update1
```

图形界面：在“软件包”输入框旁点 **“选择版本”**，输入包名后查询；
版本列表支持 Ctrl/Shift 多选，点 **“加入软件包列表”** 后正常解析/下载。

实现方式：

- 同一软件包的多个版本无法在一个 apt 事务里同时安装，工具会**每个版本单独跑一次** `apt-get install --download-only pkg=version`；
- 每个版本的依赖闭包分别解析，最终把 `.deb` 去重合并到同一个 `debs/`，生成一个 `manifest.json` 和一个包含多个版本的离线 apt 源；
- `manifest.json` 的 `version_plans` 记录每个版本请求组及其文件；
- 离线源里同时存在多个版本，目标机可用 `apt install pkg=版本` 精确安装，但 `apt` 默认候选仍按 pin/优先级选择最高版本；
- 不同版本的单次下载仍沿用当前 profile/架构/baseline/Recommends 设置；多版本合并时要求这些设置一致；

- 版本查询也会走隔离 apt root，支持 **中断**；查询结果来自 `apt-cache policy`，
  因此先 `update` 一次索引更准确。

解析与下载

```
# 只看计划，不下载
./bin/kylinpkg resolve -p sp1-2503-x86 zlib1g
./bin/kylinpkg resolve -p sp1-2503-arm64 -a arm64 --full --json zlib1g

# 下载到指定目录，并生成离线 apt 源
./bin/kylinpkg download -p sp1-2503-arm64 -a arm64 --full \
  -o ./out/arm64 --repo zlib1g

# 不指定 -o 时默认输出到 /tmp/kylinpkg-out/<profile>-<架构>
./bin/kylinpkg download -p sp1-2503-x86 zlib1g

# 只下载 zlib1g 本身，不下载依赖
./bin/kylinpkg download -p sp1-2503-x86 --no-deps \
  -o ./out/only zlib1g
```

闭包模式：

- profile 绑定了 baseline → 默认**增量**：只下载目标机缺失/需要升级的包；
- 没有绑定 baseline → 默认**全量**；
- `--full` 强制全量；`--baseline NAME` 临时指定基线；`--baseline empty` 等价于全量。

其他常用参数：

- `--no-deps`：只下载请求的包本身，**不解析也不下载依赖**（对应 GUI 取消勾选“下载依赖”）。
- `--no-recommends`：不拉 Recommends（默认与 `apt install` 一致，会拉；`--no-deps` 时无影响）。
- `--no-update`：复用 12 小时内的索引（源配置指纹变化时会自动失效）。
- `--force-update`：清空隔离索引后重新下载。
- `-a amd64,arm64`：目标架构集合；第一个是"本机架构"，决定无架构限定的依赖如何解析。

只下载当前包（不含依赖）

勾选/取消 GUI 的 **“下载依赖”**，或命令行加 `--no-deps`：

- 带依赖（默认）：`apt-get install --download-only`，下载完整依赖闭包；
- 不带依赖：改用 `apt-get download pkg=版本`，只取请求包本身，
  即使 baseline 显示已安装也会下载（适合单独归档/替换某个 `.deb`）；
- 此时 `manifest.json` 中 `with_dependencies` 为 `false`；

- 生成的离线源只含请求包；目标机安装前必须已经具备依赖，
  否则 `apt install` 仍会报缺依赖；
- 与“选择版本/多版本”可组合：

```
./bin/kylinpkg download -p sp1-2503 --no-deps -o ./out \
  zlib1g=1:1.2.11.dfsg-2kylin1.5k0.2 \
  zlib1g=1:1.2.11.dfsg-2kylin1.5k0.3update1
```

下载完成后目录结构：

```
out/arm64/
  debs/*.deb
  manifest.json          # 包名、版本、架构、大小、SHA256、来源、with_dependencies
  Packages / Packages.gz # 离线源索引
  Release
  sources.list           # 目标机可直接使用的源片段
```

`manifest.json` 是自校验清单；`kylinpkg verify DIR` 会逐包重算 SHA256，并核对 Packages 索引。

下载任务记录

每次 `kylinpkg download`（CLI 或 GUI，无论成功、失败还是中断）都会自动把任务
配置和结果记入状态目录的 `history.json`（默认保留最近 200 条）：

```
# 查询任务记录
./bin/kylinpkg history list            # 最近的任务（时间、状态、profile、包、输出目录）
./bin/kylinpkg history list --limit 20
./bin/kylinpkg history show <ID>       # 详情 + 可直接复制的等价命令

# 按历史任务的配置重新下载（输出目录、软件包、基线等全部沿用原任务）
./bin/kylinpkg history redo <ID>
./bin/kylinpkg history redo <ID> -o ./out/new-dir --force --repo

# 管理记录（不影响已下载的文件）
./bin/kylinpkg history remove <ID>
./bin/kylinpkg history clear --yes
```

- 任务 ID 支持唯一前缀缩写；
- 重做时会重新解析依赖（源里的版本可能已变化），完成后同样写入一条新记录；
- `history show` 会打印该任务的**等价命令**，复制出来改成当前任务即可——
  这就是“把历史任务的配置应用到当前”；

- 对应的 GUI 入口：顶部 **“任务记录”** 按钮，可查询、**重新下载**、**应用到当前参数**（自动填表：profile/基线/架构/软件包/输出目录/选项）和删除记录。

目标机怎么用

方式一：离线 apt 源

```
# 把整个目录拷到目标机，例如 /opt/kylinpkg/arm64
# 修改 sources.list 中的路径：
# deb [trusted=yes] file:/opt/kylinpkg/arm64 ./
sudo cp sources.list /etc/apt/sources.list.d/kylinpkg-local.list
sudo apt update
sudo apt install zlib1g
```

方式二：直接装 deb

把 `debs/*.deb` 拷到目标机同一目录，用你现有的 `deb-install.run` 双击安装即可（它会调 `dpkg -i` + `apt-get -f install`）。

常见失败原因（麒麟实测）

现象	原因	处理
`libc6` 没有候选	`libc6` 在 **universe**，源里只挂了 main	四个组件都挂上
`libgcc-s1` 无法安装	只挂了叠加 suite，缺基础 suite	同时挂 `10.1-kylin`
解析出的版本和目标机不同	没带 `preferences`（pin）	`--from-system` 或 `--preferences-*` 导入
`NO_PUBKEY` / 签名失败	keyring 没导入	`--keyring`、`--from-system`，或源加 `trusted=yes`
`does not have a Release file`	suite 名称不对	对照目标机 `apt-cache policy` 或源文件
增量模式直接报架构不一致	baseline 有 i386/arm64 等额外架构	用 `-a` 补齐，或换匹配的 baseline
root 运行时下载失败	`/home` 权限 700，apt 的 `_apt` 沙箱进不去	工具已自动加 `APT::Sandbox::User=root`
SSH 抓取失败	`ssh` 需要交互输密码	先配置 SSH key；工具使用 `BatchMode=yes`
`apt update` 长时间卡住	IPv6 路由或第三方源不可达	点 GUI 的“中断”终止任务；工具默认 `Acquire::ForceIPv4=true`；仍卡住就编辑源去掉不可用地址；纯 IPv6 环境可在 profile 的 `apt_options` 里覆盖

状态目录与安全

```
~/.local/share/kylinpkg/
  profiles/<name>/       # sources、preferences、keyrings、profile.json
  baselines/<name>/      # status、meta.json
  roots/<profile>/<arch>/# 隔离 apt root（可随时 gc）
  downloads/             # 预留
```

- 目录 0700，文件 0600；源里若内嵌凭据，`profile show` 默认脱敏，`--raw` 才显示原文。
- 下载由 apt 自身完成并校验哈希；`file:` 源由工具复制并复核。
- 离线源默认写 `[trusted=yes]`，生产环境建议改用签名源或把 keyring 一并下发。
- `kylinpkg gc --cache` 可清理可重建的隔离索引/包缓存。

清理缓存

图形界面顶部有 **“清理缓存”** 按钮（“文件”菜单里也有）。命令行：

```
# 清理全部 profile 的隔离 apt 索引和包缓存
./bin/kylinpkg gc --cache

# 只清理某个 profile
./bin/kylinpkg gc --cache --profile sp1-2503
```

清理内容：

- `~/.local/share/kylinpkg/roots/` 下的 apt 包索引（lists）、apt archives 缓存、pkgcache、partial 下载等可重建数据；
- **不会删除** profile、baseline、已下载到输出目录的 `.deb` /manifest/离线源；
- 清理后会显示释放的空间；下次解析会自动重建索引。

`--roots` 是 `--cache` 的别名。任务运行期间 GUI 按钮会自动置灰，避免误删正在使用的索引。

测试

```
# 单元测试（不联网；有 DISPLAY 时会顺带做 GUI 冒烟测试）
# test_bundle 会实际运行 collector 并导入临时状态；
# test_versions 用本地双版本 deb 仓库验证版本查询、多版本合并和离线源回读；
# test_no_deps 验证“仅当前包”与完整依赖两种模式；
# test_cache 验证缓存清理不会动 profile/baseline
python3 -m unittest discover -s tests -v

# 联网集成测试：arm64 全量闭包、下载、离线源回灌、增量零下载、宿主未被修改
KYLINPKG_NET_TESTS=1 python3 -m unittest tests.test_integration -v
```

边界

- 镜像上旧版本包可能被删除；需要精确复现时应在 profile 中固定 suite，必要时配合快照源（本版未实现）。
- 跨架构只做依赖求解和下载，不执行目标架构二进制；`--download-only` 不需要 qemu。
- 第三方源的 `Signed-By: /绝对路径` 在本机可用；跨机器复用时需要把对应 keyring 一并导入。
- 依赖求解使用的是**运行工具这台机器上的 apt 版本与解析器**。目标机与工具机跨大版本（例如 SP1 与 SP3）时，建议在对应版本的系统上运行，或至少用同版本基线核对结果。

许可证

本项目采用 MIT 许可证，`Copyright (c) 2026 mohao`。

- 允许自由使用、复制、修改、合并、发布、分发、再许可和销售副本，包括用于商业目的；

- 唯一义务是**在所有副本或实质性部分中保留版权声明和许可声明**；
- 软件按“原样”提供，不含任何明示或隐含的担保，作者不对任何索赔或损害负责；
- 每个源文件的头部都带有 `SPDX-License-Identifier: MIT` 标识。

商标与免责

- 本项目是独立的第三方工具，**不是麒麟软件有限公司的官方产品**，未获得其授权、赞助或背书。
- “麒麟”“Kylin”及相关商标归各自权利人所有。本项目仅在描述性意义上使用这些名称（用于说明本工具所面向的操作系统与软件源），MIT 许可**不包含任何商标授权**。
- 本工具从各软件源下载、生成的 deb 包与离线源，其权利与许可仍归各自软件的权利人所有，与本项目的许可无关；请遵守对应软件自身的许可条款。